Algebraic Effects with Non-Recoverable Failure

Timotej Tomandl
University of Kent

Dominic Orchard
University of Kent

Abstract

Algebraic effects are one of the possible approaches to tackle modularity issues arising in applications where normally we would use monad transformers. One such area is programming language semantics, we show that the standard incarnation of effects does not satisfy some requirements needed to build compact, non-repetitive semantics, when the language does not have a type system such as in cases of intermediate languages in compilers. We show how to use a single MaybeT transformer to achieve this, we also explore the graded structure of effect monads.

1 Motivation

Consider the following situation. A compile writer is producing some semantics-preserving translations between intermediate languages. At some point in this transformation pipeline, information is lost, e.g., typing information is erased when converting to an untyped core lanugage or byte code. This necessitates that the compiled code has some error handling cases, for example:

```
pseudo code
```

This failure is intentionally non-recoverable. In Haskell, this idea is captured by the `MonadFail` type class which is used to desugar the *do*-notation in the case of refutable patterns. For example

Dom: Show translation

In this work, we seek to capture the same idea but in the setting of *algebraic effects*.

One aims of the algebraic-effects approach is to allow the management of side effects to be *extensible*, considering first the necessary effectful operations and their laws, rather than focussing on the underlying semantic structure as is common in the *monad*-first approach. *Monad transformers* can provide some modularity to the monad-first approach, but this needs a new type class instance for every combination of effects and this makes the implementation less flexible. Dom: This is a very Haskell answer... is there something better here.

Our results. We have produced an intermediate construction between the variations of Kiselyov in Agda. This construction is inbetween the construction the monads Eff and Freer as presented in their work. This is due to our focus on reasoning about monadic laws and ease of construction within a dependently-typed framework instead of performance. We further describe how to extend these with failing pattern matching, for simpler description, we present our code in stages and explain how to generically add TODO add short explanation of similarities and differences Dom: Might be better to do the similarities and differences inline as we go.

2 Algebraic effects via graded monads

Algebraic effects in theory consist of a signature of operations and their equations. An effect is represented by a type family handling particular effect together with a name identifying the given effect. Dom: Too Haskell.. can we say in a more general way.

Dom: Oleg's definition

Dom: Then ours

When the effect set is empty we should be able to extract the final value by realizing that there can be no proof of membership within a set.

Algebraic effects as monads and graded monads. Dom: define graded monads briefly, probably abstractly in mathematics

Algebraic effects can be presented on top of a monadic interface, we provide this interface by implementing it using a free monad. The importance of this interface is that we can provide an interface, which chains different operations together and allows us to give a correct interface. Dom: this sentence uses the word interface three times!

For graded monads this allows us to do an interface, which combines different effect sets.

2.1 Effect monad construction

We review the construction of Effect monad, to illustrate what issues pop up, while trying to make this into a admissible construction in Agda. In haskell, these constructions can be done easily using the general datatypes, but they included datatype level fixed points in full. In agda we can take 2 possible views of these constructions. We could go through the containers and construct a fixed point of containers as in [?]. Or we could fuse the left kan extension construction into the effect monad and observe that the result is strictly positive. This means we do not have to provide a container construction for the underlying work as in [?]. This works by realizing that the instantiation of F in the recursion to Lan of an arbitrary datatype is strictly positive without a recourse to containers. We pick to do a halfway point between the constructions in Kyseliov, we do not use the optimized version, but we use the idea of effects should belong to a set.

2.2 Graded monad proofs

Dom: is the intention here to explain about how we did the mechanisation? We grade monads to allow to combining the

effect sets and thus being able to refer to only subcomputations in the programs and proofs. We do this by describing a reindexing operation, which combines sets and allows us to refer to distinct effects with same names. We need a way to reindex the effect set to embed the left effect set into a list. Dom: the following sounds like it should go above in the definition of graded monads / the instance we are defining here. We note that the list of effects is itself a monoid and we grade by it. Graded unit, is the unit of our monad but for empty lists. To produce graded bind, we append the 2 lists of effects and combine them. We

Handling effects. Algebraic effects provide ways to write programs causing effects which are treated abstractly initially but which are given concrete implementations afterwards using *handlers* (akin to raising an exception and later handling it). This generalizes to general effects: effect handlers provide a way to not only handle general exceptions, but to also remember some state between handling different operations a handler is responsible for. Handlers are then also constrained to respect a set of domain-specific equations. Dom: How much do we need to get into handlers here?

3 Leveraging MaybeT and MonadFail

In Haskell, if a *do*-expression performs a pattern matching 'bind' that may fail, then the monad being used must also have an instance of the `MonadFail` class:

```
class MonadFail m where
  fail :: String -> m a
```

Dom: link with MaybeT MaybeT is the monad transformer, i.e. a transformation $TA : Set- > MonadT- > MaybeTA- > MonadS$. $MaybeTmx = m(Maybex)$ We can think of MaybeT as an extension by an arbitrary, single option failure case. especially in the strucutre of a Free monad as Monad is Pure and fixed point of a functor with this identity case.

3.1 Handling EffectFail

We have to modify handling, we state a different form of initiality, as much as effects are inspired by initiality, we provide a type-theoretic motivation for our definition, to provide handling of effectfail, we interpret the results and embed them into Maybe and propagate the underlying failure. This should be correct not by some appeal to category theory as initiality is normally stated, but by appealing to the API we would expect from our system. We would excpect this API to propagate pattern matching failure. To run EffectFail we either want to extract the underlying result or propagate the failure of underlying Pure result.

4 Future directions

As much we have constructed the monad in agda and verified the various laws. For practical purposes there needs to be more than just a monadic structure. We want to reason about the local code using the eponymous algebraic laws. We would like to attach equations the handlers should satisfy to the monad. There are multiple interesting directions to take this in. We would also like to use Quotient inductive types to provide a view of operations for reasoning and correct-by-construction definition of handlers, which abstracts away from the concrete choices when reasoning, yet forces us to prove satisfiability of the algebriac equations by some handler. To extend the proofs from our current work to the quotient case we need to prove a type-theoretic formulation of algebraic freedom [?]. To verify that all the constructions preserve all equations. We note that by theorem 3.1 and 3.2 [?] this result should follow in Homotopy type theory. The reason for this being that we need the ambient category to be locally small and complete with homotopy type theory we should be able to get all colimits.

Another direction of future research is to construct an isomorphism between continuations and the fast type-aligned queues of Kiselyov, quotiented by their composition, thus providing a more efficient implementation of effects without the overheads to reasoning this entails as we focus on semantic constructions instead of executable code, this is not our primary concern. We would expect this to then transport by univalence, but as discussed in unitransport we would need also some form of representation independence coming from parametricity for this to be actually efficient.

One of our larger aims is to explore how to emulate modular monadic semantics [Delaware et al. 2013], which provides composable way to combine language feature with monadic semantics approach, which allows expressing semantics in terms of operations provided by transformers. We hope this gives a good start towards managing complexities, which arise with pattern matching and algebraic effects.

A Appendix Title

This is the text of the appendix, if you need one.

References

Benjamin Delaware, Steven Keuchel, Tom Schrijvers, and Bruno C.d.S. Oliveira. 2013. Modular Monadic Meta-Theory. *SIGPLAN Not.* 48, 9 (Sept. 2013), 319–330. https://doi.org/10.1145/2544174.2500587

Oleg Kiselyov, Amr Sabry, and Cameron Swords. 2013. Extensible Effects: An Alternative to Monad Transformers. In *Proceedings of the 2013 ACM SIGPLAN Symposium on Haskell* (Boston, Massachusetts, USA) *(Haskell '13)*. Association for Computing Machinery, New York, NY, USA, 59–70. https://doi.org/10.1145/2503778.2503791